

LAB 05 Python c.d.

*Zakład Mechaniki Budowli i Zastosowań Informatyki
Wydział Inżynierii Lądowej
Politechnika Warszawska*

Koordynator przedmiotu: dr hab inż. Sławomir Czarnecki

Zad. 1. Zaimplementuj rekurencyjny algorytm poszukiwania binarnego składowej x w posortowanej liście a o wymiarze $n = \text{len}(a)$ jako funkcję $\text{binary_rek}(a, \text{first}, \text{last}, x)$, dla której w chwili wywołania: $\text{first} = 0$, $\text{last} = n - 1$.

Funkcja $\text{binary_rek}(a, \text{first}, \text{last}, x)$ ma zwracać indeks i ($0 \leq i < n$), dla którego $x = a[i]$ lub ma zwracać wartość -1 , jeśli nie istnieje taki i , dla którego $a[i]$ jest równy x .

Algorytm rekurencyjny poszukiwania binarnego

elementu x w n wymiarowej, posortowanej liście a implementowany jako funkcja $\text{binary_rek}(a, \text{first}, \text{last}, x)$, gdzie w chwili wywołania: $\text{first} = 0$, $\text{last} = n - 1$.

Zakładamy, że lista a **jest posortowana** od najmniejszej do największej wartości.

- **Oblicz** indeks: $\text{middle} = (\text{first} + \text{last}) / 2$ (dzielenie całkowito liczbowe)
- **Jeśli** $\text{first} > \text{last}$ **to zwróć** wartość -1 (brak elementu x w liście a)
- **w przeciwnym przypadku jeśli** $x = a[\text{middle}]$ **to zwróć** indeks middle ,
- **w przeciwnym przypadku jeśli** $x < a[\text{middle}]$ **to szukaj** elementu x w lewej połowie listy a od indeksu first do indeksu $\text{middle} - 1$ czyli **zwróć wartość** zwracaną przez funkcję $\text{binary_rek}(a, \text{first}, \text{last}, x)$ dla indeksu $\text{last} = \text{middle} - 1$
- **w przeciwnym przypadku szukaj** elementu x w prawej połowie listy a od indeksu $\text{middle} + 1$ do indeksu last , czyli **zwróć wartość** zwracaną przez funkcję $\text{binary_rek}(a, \text{first}, \text{last}, x)$ dla indeksu $\text{first} = \text{middle} + 1$.

Uwaga ! Obliczanie indeksu middle należy przeprowadzać na podstawie operatora dzielenia całkowitoliczbowego „/”.

Przetestuj działanie funkcji $\text{binary_rek}(a, \text{first}, \text{last}, x)$ dla posortowanej listy.

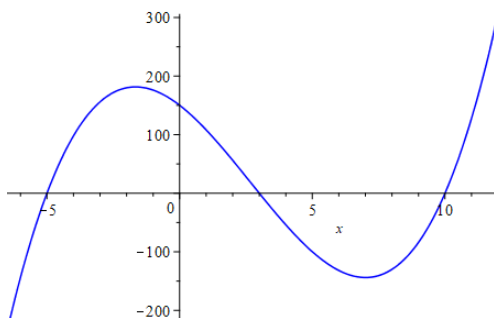
Zad. 2. Zdefiniuj funkcję $bisection_rek(xlewy, xprawy, eps, f)$ implementując rekurencyjny algorytm metody bisekcji poszukiwania pierwiastka ciągłej funkcji f w przedziale domkniętym $[xlewy, xprawy]$ z dokładnością $eps > 0$. **Zakładamy**, że **wartości funkcji f na końcach tego przedziału są różne od zera i mają różne znaki**. Funkcja $bisection_rek(...)$ ma zwracać pierwiastek funkcji f .

Algorytm metody bisekcji w wersji rekurencyjnej

- Rozpoczynamy od wyznaczenia **punktu środkowego** $xsrodek = (xlewy + xprawy) / 2$.
- **Jeśli** $|f(xsrodek)| < eps$ **lub** $|xprawy - xlewy| < eps$ **to zwróć** pierwiastek jako $xsrodek$.
- **w przeciwnym przypadku jeśli** $f(xlewy) \cdot f(xsrodek) < 0$ **to zwróć** wartość zwracaną przez funkcję $bisection_rek(xlewy, xprawy, eps, f)$ dla $xprawy = xsrodek$
- **w przeciwnym przypadku zwróć** wartość zwracaną przez funkcję $bisection_rek(xlewy, xprawy, eps, f)$ dla $xlewy = xsrodek$

Przyjmujemy, że zwracana z funkcji $bisection_rek(xlewy, xsrodek, eps, f)$ wartość zmiennej $xsrodek$, jest poszukiwanym pierwiastkiem, tj. że $f(xsrodek) \approx 0$. Przetestuj funkcję $bisection_rek(xlewy, xprawy, eps, f)$ poszukiwaniu pierwiastka funkcji f w przedziałach $[-6, -4]$, $[0, 5]$, $[8, 11]$ dla wartości $eps = 10^{-7}$, gdzie

$$f(x) = x^3 - 8x^2 - 35x + 150$$



Zad. 3. Zdefiniuj rekurencyjną funkcję $binomial_rek(n, k)$ obliczania współczynnika dwumianowego

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad \text{dla } 0 \leq k \leq n$$

na podstawie prostej do udowodnienia zależności rekurencyjnej

$$\binom{n}{k} = \begin{cases} 1 & \text{jeśli } k = 0 \text{ lub } k = n \\ \binom{n-1}{k-1} + \binom{n-1}{k} & \text{w przeciwnym przypadku} \end{cases}$$

eliminującej konieczność obliczania wartości wyrażeń typu $n!$ lub $k!$.

Algorytm rekurencyjny oblicza aż $2\binom{n}{k} - 1$ wyrazów w celu obliczenia wartości $\binom{n}{k}$.

Zad. 4. W pierwszym wariancie zdefiniuj funkcję *Fibonacci_rek(kmax)* wyświetlającą w oknie konsoli k_{max} pierwszych wyrazów F_k , $k = 1, 2, 3, \dots, k_{max}$ ciągu Fibonacciego obliczanych według następującej reguły rekurencyjnej: dwa pierwsze F_1 , F_2 wyrazy są równe 1, a kolejne F_k dla $k = 2, 3, \dots$ obliczamy z rekurencyjnej zależności: $F_k = F_{k-1} + F_{k-2}$, tj.

$$F_k = \begin{cases} 1, & k = 1, 2 \\ F_{k-1} + F_{k-2}, & k \geq 3 \end{cases}$$

$$\begin{array}{rcl} F_1 & = & 0 + 1 \\ F_2 & = & 1 + 0 \\ F_3 & = & F_2 + F_1 \\ F_4 & = & F_3 + F_2 \\ \dots & & \dots \\ F_{k-2} & = & F_{k-3} + F_{k-4} \\ F_{k-1} & = & F_{k-2} + F_{k-3} \\ F_k & = & F_{k-1} + F_{k-2} \end{array}$$

$$F_k^{old} = F_{k-1}^{old} + F_{k-2}^{old}$$

$$F_k^{new} = F_{k-1}^{new} + F_{k-2}^{new}$$

Dla utrudnienia, w wariancie tym, w kodzie funkcji *Fibonacci_rek(kmax)* nie może być używana żadna struktura sekwencyjna oprócz łańcucha znaków czyli struktury typu *str*. Kolejne wyrazy ciągu Fibonacciego muszą być dopisywane do łańcucha znaków natychmiast po ich obliczeniu według reguły rekurencyjnej, za **każdym razem w nowej linii**. Znakiem nowej linii jest znak '\n'. Przykładowo, dla $k_{max} = 30$ wydruk ciągu Fibonacciego na ekranie konsoli powinien wyglądać następująco:

```
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
1597
2584
4181
6765
10946
17711
28657
46368
75025
121393
196418
317811
514229
832040
```

Poniżej **opis alternatywnego algorytmu rekurencyjnego** obliczania kolejnych wyrazów F_k ciągu Fibonacciego nie przekraczających wartości $F_{\max}$:

$$a = 0$$

$$F_k = 1$$

dopóki dopóty $F_k < F_{\max}$

F_k jest kolejnym wyrazem ciągu Fibonacciego

$$b = a + F_k$$

$$a = F_k$$

$$F_k = b$$

Zaimplementuj powyższy algorytm w funkcji *fib(Fmax)* w taki sposób, aby tym razem po jej wywołaniu **wyrazy ciągu były wyświetlane** w oknie konsoli **w jednym wierszu**. Sprawdź poprawność jego implementacji dla $F_{\max} = 832041$.

W drugim wariancie zdefiniuj nową funkcję *Fibonacci_iter(kmax)*, która po uruchomieniu ma wyświetlać w oknie konsoli $k_{\max}$ pierwszych wyrazów F_k , $k = 1, 2, 3, \dots, k_{\max}$ ciągu Fibonacciego obliczanych tym razem według następującej reguły iteracyjnej:

$$F_k = \frac{1}{\sqrt{5}} \left[\left(\frac{1+\sqrt{5}}{2} \right)^k - \left(\frac{1-\sqrt{5}}{2} \right)^k \right], \quad k \geq 1 .$$

Tak jak i w pierwszym wariancie, w kodzie funkcji *Fibonacci_iter(kmax)* nie może być używana żadna struktura sekwencyjna oprócz łańcucha znaków czyli struktury typu *str*. Sprawdź czy testowe wyświetlenie wyrazów ciągu Fibonacciego będzie identyczne jak w pierwszym wariancie.

W trzecim wariancie, zmieniając nieznacznie rekurencyjną formułę definiowania ciągu Fibonacciego na następującą (wyrazy ciągu w tej nowej wersji liczone będą nie od 1, ale od 0, tak jak we wszystkich strukturach sekwencyjnych):

$$F_0 = 0$$

$$F_1 = 1$$

...

$$F_k = F_{k-1} + F_{k-2} \quad \text{dla } k = 2, 3, \dots$$

możemy natychmiast napisać, tym razem w pełni rekurencyjną (w sensie semantyki dowolnego języka programowania) wersję funkcji *fibonacci(k)*

```
def fibonacci(k):
    if k == 0:
        return 0
    elif k == 1:
        return 1
    else:
        return fibonacci(k - 1) + fibonacci(k - 2)
```

obliczania dowolnego k -tego elementu tego ciągu, dla $k = 0, 1, 2, \dots$. **Funkcja ta** (dostępna w pliku) **nie oblicza wszystkich wyrazów ciągu** do np. k -tego wyrazu włącznie, ale **potrafi** jedynie **obliczyć** dowolny, k -ty **element tego ciągu**.

Wykorzystując rekurencyjną funkcję $fibonacci(k)$ napisz kod pozwalający obliczyć składowe listy o nazwie FIBONACCI przechowującej wszystkie wyrazy tego ciągu od indeksu 0 do indeksu $kmax$ włącznie, po czym na ekranie konsoli wyświetl wszystkie składowe tej listy od 1 do $kmax$, aby uzyskać potwierdzenie, że obliczone wyrazy tego ciągu są identyczne jak w poprzednich wariantach.

Zad. 5. Algorytm wieży z Hanoi

Założenia: n = liczba krążków (dysków) (zakładamy, że $n > 0$)

- Wieże F,A,T będą reprezentowane przez listę liczb naturalnych
- Krążek o największej średnicy ma przyporządkowaną liczbę n .
- Krążek o najmniejszej średnicy ma przyporządkowaną liczbę 1.
- Pozostałe krążki są oznaczone przez numery od 2 do $n - 1$ w zależności od ich średnic.

Stan początkowy:

$F(n-1)=1$	$A(n-1)=0$	$T(n-1)=0$
$F(n-2)=2$	$A(n-2)=0$	$T(n-2)=0$
$F(n-3)=3$	$A(n-3)=0$	$T(n-3)=0$
...	...	...
$F(1)=n-1$	$A(1)=0$	$T(1)=0$
$F(0)=n$	$A(0)=0$	$T(0)=0$

$F(i)=0$, $A(i)=0$ lub $T(i)=0$ ($i = 0, 1, \dots, n-1$) oznacza tutaj puste miejsce – brak krążka.

Stan końcowy:

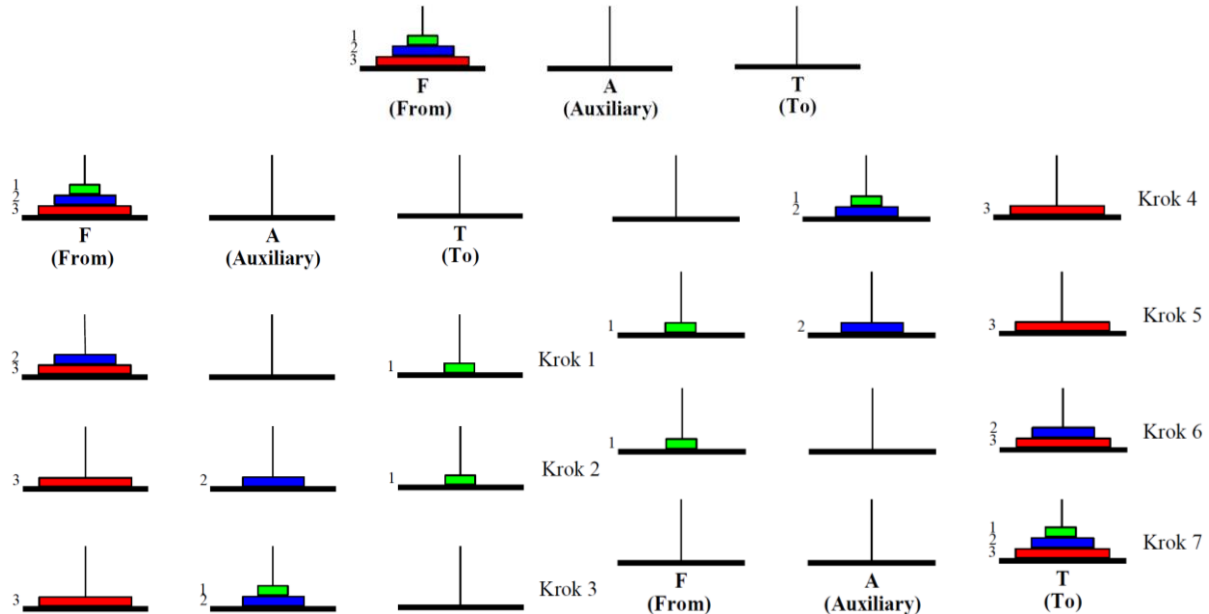
$F(n-1)=0$	$A(n-1)=0$	$T(n-1)=1$
$F(n-2)=0$	$A(n-2)=0$	$T(n-2)=2$
$F(n-3)=0$	$A(n-3)=0$	$T(n-3)=3$
...	...	...
$F(1)=0$	$A(1)=0$	$T(1)=n-1$
$F(0)=0$	$A(0)=0$	$T(0)=n$

Klasyczny problem rekurencyjny: Wieże z Hanoi

• Problem (nie matematycznej natury) dotyczy krążków ułożonych w wieży (powiedzmy F) jeden na drugim, od największego na samym dole do najmniejszego na samej górze, które przy dysponowaniu dwoma dodatkowymi i identycznymi wieżami (powiedzmy A i T), należy przenieść na jedną z nich (powiedzmy na wieżę T) stosując następujące trzy reguły:

1. za każdym razem można przenosić tylko jeden krążek
2. krążek wolno pobierać z wieży wyłącznie z samej góry i podobnie wolno kłaść go na samą górę w innej wieży
3. przy każdym przenoszeniu krążka, nie można dopuścić do sytuacji aby krążek o większej średnicy leżał na krążku o mniejszej średnicy

Dysponując zatem tylko trzema wieżami (F, A, T), musimy stosując powyższe trzy reguły przenieść dowolną ustaloną liczbę krążków z pierwszej wieży F na trzecią wieżę T, korzystając dodatkowo tylko z drugiej wieży pomocniczej A.



ALGORYTM HANOI dla dowolnej liczby n krążków

Jeśli $n = 1$ (przypadek elementarny) to przenieść (jedyny) dysk 1 z FROM na TO.

Jeśli $n \geq 2$ to założmy, że potrafimy przenieść $n - 1$ dysków z dowolnej wieży na inną dowolną wieżę (założenie indukcyjne). Wtedy nasz problem może być zredukowany do 3 „mniej złożonych” zadań:

Zad 1: na mocy założenia indukcyjnego przenieść górne $n - 1$ dysków z FROM na AUXILIARY używając TO jako wieżę pomocniczą. Na AUXILIARY będziemy mieli zatem wszystkie górne $n - 1$ dysków z FROM, natomiast na FROM pozostanie tylko największy dysk n .

Zad 2: przenieść największy i jedyny dysk n z wieży FROM na pustą wieżę TO – przypadek elementarny. Teraz wieża FROM będzie pusta, a na wieży TO leży największy i póki co jedyny dysk n .

Zad 3: na mocy założenia indukcyjnego przenieść wszystkie $n - 1$ dysków z AUXILIARY na TO używając FROM jako wieżę pomocniczą i ostatecznie na TO będziemy mieli wszystkie n dyski.



Pod-problem 1: przenieść górne $n - 1$ dysków z FROM na AUXILIARY używając TO jako wieżę pomocniczą
(założenie indukcyjne)

Pod-problem 2: przenieść największy dysk n z FROM na TO
(przypadek elementarny)

Pod-problem 3: przenieść wszystkie $n - 1$ dysków z AUXILIARY na TO używając FROM jako wieżę pomocniczą
(założenie indukcyjne)

Zaimplementuj rekurencyjny algorytm wieży Hanoi w procedurze

hanoi(*FROM*, *AUXILIARY*, *TO*, *n*), gdzie *FROM*, *AUXILIARY*, *TO* oznaczają kolejno wieże: *FROM*, *AUXILIARY*, *TO* zdefiniowane jako jednowymiarowe listy liczb naturalnych – dokładniejsze wyjaśnienia będą podane na laboratorium.

De-rekursywacja – wiele algorytmów rekurencyjnych można obecnie przedstawić w ich wersji iteracyjnej, stosując pewne standardowe i opracowane wyłącznie w tym celu techniki algorytmiczne.